

HISABDO AI

DAY 12 — AI SPENDING ANOMALY DETECTION

Professional MVP Specification, Detection Rules & .NET Implementation Guide

1. Document Purpose

This document defines the Day 12 AI Spending Anomaly Detection module for HisabDo AI. Using verified, user-scoped transaction history, the module identifies spending that materially differs from the user's normal pattern and produces structured anomaly results for the AI explanation layer.

2. Business Objective

HisabDo AI should proactively alert users when spending looks unusual instead of only recording transactions. The MVP covers four anomaly types: Unusual Amount, Sudden Spending Spike, Unusual Category, and Duplicate Transaction.

- Record → Understand → Analyze → Detect → Explain → Recommend → Action
- The detection engine determines the anomaly; the LLM explains verified results.
- An anomaly is not proof of fraud.

3. Scope

- Retrieve only authenticated user's permitted expense transactions.
- Build a historical spending baseline.
- Detect unusual transaction amounts.
- Detect sudden current-period spending spikes.
- Detect unusual category behavior.
- Detect likely duplicate transactions.
- Return deterministic severity, reason codes and verified values.
- Provide results to AI, mobile and web applications.
- Keep financial calculations outside the LLM.

4. Required HisabDo Data

Field	Priority	Purpose	Detection Use
UserId	Required	User isolation	All
TransactionId	Required	Unique transaction	Duplicate
TransactionType	Required	Income/expense separation	All
Amount	Required	Financial comparison	Amount/spike/category
Category	Required	Category baseline	Category/spike
Date	Required	Period/history comparison	All
Description/Note	Recommended	Merchant/duplicate support	Duplicate
Customer/Person	Optional	Future context	Future enhancement

The project brief defines users, transactions, dates, amounts and categories as core AI-ready financial data. Day 12 additionally requires enough historical transaction data to establish a user-specific baseline.

5. Analysis Period & Historical Baseline

- Current period is the period being analyzed, such as the current calendar month.
- Historical baseline should use previous completed comparable periods.
- MVP target: at least 3 comparable historical periods when available.
- If history is insufficient, return an insufficient-baseline state rather than inventing a normal pattern.
- All baselines and comparisons must be calculated per authenticated user.

6. What Qualifies as an Unusual Expense?

An expense is unusual when its verified amount, category behavior, period spending or duplication pattern materially differs from the user's established historical behavior. The MVP uses deterministic, configurable rules so every flag can be explained and tested.

7. Anomaly Types

7.1 Unusual Amount

- Compare a current transaction with historical amounts for the same category when sufficient history exists.
- Preferred MVP rule: $\text{Amount} \geq \max(\text{Category Average} \times 2.0, \text{Category Average} + 2 \times \text{Standard Deviation})$.
- If category history is insufficient, an overall expense baseline may be used only when sufficient history exists.
- Flagged result should use 'unusual' language; it must not claim fraud.

7.2 Sudden Spending Spike

- Calculate current-period expense total.
- Calculate historical comparable-period average.
- $\text{Growth \%} = ((\text{Current} - \text{Historical Average}) / \text{Historical Average}) \times 100$.
- 20–49.99% increase → Medium severity.
- ≥50% increase → High severity.
- If historical average is zero, do not divide; return insufficient-baseline information.

7.3 Unusual Category

- Compare current category spending with its historical category baseline.
- MVP rule: $\text{Current Category Amount} \geq \text{Historical Category Average} \times 2.0$.
- A new category may be reported as 'New Category'; do not automatically label every new category as anomalous.
- The threshold must remain configurable and be validated on real HisabDo data.

7.4 Duplicate Transaction

- Same authenticated user.
- Expense transaction type.
- Same amount in the MVP.
- Same category when both are available.
- Same/normalized description when both are available.
- Same calendar day in the MVP.
- Return a duplicate candidate; never automatically delete or reverse the transaction.

8. MVP Thresholds

Rule	Default threshold	Severity	Result
------	-------------------	----------	--------

Unusual Amount	$\geq 2 \times$ category average OR $\geq$ average + 2σ	High	Flag
Spending Spike	20–49.99%	Medium	Flag
Spending Spike	$\geq 50\%$	High	Flag
Unusual Category	$\geq 2 \times$ category historical average	Medium/High	Flag
Duplicate	Same amount/category/description on same day	Medium	Candidate

These are MVP engineering defaults, not fraud-detection or banking-regulatory thresholds. They must be configurable and validated against real HisabDo data.

9. Comparison Logic

- Filter to expense transactions for the authenticated user.
- Build overall and category-level historical baselines.
- Calculate current-period totals and category totals.
- Compare current values with historical baselines.
- Calculate percentages only when the denominator is greater than zero.
- Keep exact verified values in the API payload for the AI layer.

10. Severity Model

Severity	Meaning	Example
Low	Minor deviation to monitor	Small deviation
Medium	Meaningful deviation	Category spending doubled
High	Large deviation	Expense $\geq 3 \times$ normal or spike $\geq 50\%$

11. Verified API Result

Recommended contract: GET /api/ai/anomalies/{userId}?startDate=YYYY-MM-DD&endDate=YYYY-MM-DD

Example:

```
{
  "userId": "verified-user-id",
  "analysisPeriod": {"start": "YYYY-MM-DD", "end": "YYYY-MM-DD"},
  "baselinePeriods": 3,
  "anomalies": [{
    "transactionId": "verified-id",
    "type": "UnusualAmount",
    "severity": "High",
    "amount": 25000,
    "category": "Shopping",
    "date": "YYYY-MM-DD",
    "baselineValue": 9000,
    "deviationPercent": 177.78,
    "reasonCode": "AMOUNT_ABOVE_BASELINE",
    "isDuplicateCandidate": false
  }],
  "periodSpikes": [{
    "category": "Food",
    "currentAmount": 18000,
    "historicalAverage": 13000,
```

```
"growthPercent": 38.46,  
"severity": "Medium"  
}  
}
```

12. AI Explanation Requirements

- LLM receives only verified backend anomaly results.
- LLM may explain type, amount, category, baseline and verified deviation.
- LLM must not invent amounts, dates, categories, causes or fraud claims.
- LLM must not access the database directly.
- If verified data is unavailable, it must say so.
- Responses should be concise and follow the user's supported language.
- Recommendations must remain consistent with verified results.

This preserves the project Golden Rule: HisabDo Database → Calculation/Analytics Engine → Verified Result → LLM → User.

13. End-to-End Architecture

HisabDo Database → .NET Backend → User-Scoped Retrieval → Historical Baseline → Anomaly Detection Engine → Verified Anomaly DTO → AI Explanation Service → Flutter/Web

14. .NET Implementation

- Repository loads only user-scoped expense transactions.
- Service validates positive amounts and valid date ranges.
- Baseline and anomaly rules are deterministic.
- DTO contains verified values and reason codes.
- Controller must enforce authentication/authorization.
- AI provider is kept outside the calculation engine.

15. Testing & Edge Cases

1. TC-01: No transactions
2. TC-02: Only one transaction / insufficient history
3. TC-03: Zero historical spending
4. TC-04: Zero current-period spending
5. TC-05: Invalid/negative amount
6. TC-06: Income mixed with expenses
7. TC-07: Missing category
8. TC-08: New category
9. TC-09: Very large transaction
10. TC-10: Exact duplicate-looking transactions
11. TC-11: Same amount but legitimate different transactions
12. TC-12: Same description on different dates
13. TC-13: Multiple users with identical transactions
14. TC-14: Wrong historical period selection
15. TC-15: Spike exactly at 20%
16. TC-16: Spike exactly at 50%
17. TC-17: AI prompt attempts to change verified values

16. Accuracy Validation

- Create labeled normal and anomalous sample datasets.
- True Positive: anomaly correctly detected.
- False Positive: normal transaction incorrectly flagged.
- False Negative: anomaly missed.
- True Negative: normal transaction correctly ignored.
- Review precision/recall where labeled data is available.
- Tune thresholds after reviewing false positives and false negatives.
- Never interpret a single anomaly as proof of fraud.

17. Team Responsibilities

Member	Role	Deliverables
Taha	Team Lead	Requirements, rules, thresholds, architecture, review, testing, accuracy validation, documentation, approval.
Jaffer	Backend / Data Processing	User-scoped retrieval, baseline, anomaly service, DTO/API, authorization, unit tests.
Omesha	AI/ML / Anomaly Logic	Detection methodology, threshold validation, sample datasets, accuracy analysis.
Aarti	AI Explanation	Verified prompt/response format, explanation rules, hallucination prevention, samples.
Laiba	Frontend / UI	Alerts UI, severity/reason/details, API integration, empty/no-anomaly states.

18. Team Lead Acceptance Checklist

- ☐ Requirements approved
- ☐ Four anomaly types scoped
- ☐ Thresholds documented/configurable
- ☐ Historical baseline documented
- ☐ User isolation verified
- ☐ Verified DTO/API defined
- ☐ AI receives verified values only
- ☐ No fraud claims from anomaly alone
- ☐ Duplicate detection treated as candidate
- ☐ Edge cases covered
- ☐ Sample data validated
- ☐ False positives/negatives reviewed
- ☐ Mobile/Web contract clear
- ☐ Documentation complete

19. Implementation Limitation / Integration Note

The accompanying C# file is a framework-independent .NET reference implementation. Your actual HisabDo DbContext/entity/repository names and authentication implementation were not included in the source documents

available for this task. Therefore, the code uses IExpenseTransactionRepository as an integration boundary. Before production merge, map it to the real HisabDo data model and preserve the existing authorization conventions.

20. Final Definition

Day 12 Anomaly Detection is a deterministic, user-scoped intelligence module that compares current spending with verified historical behavior and flags unusual amounts, sudden spikes, unusual categories and duplicate candidates. The backend determines the verified anomaly; the LLM explains it. The system must never allow the LLM to invent financial facts and must not treat an anomaly as automatic proof of fraud.